

Dataset: Daily weather observations from Raleigh-Durham International Airport (NOAA
GHCN-Daily, 2022 - 2026)
Station: USW00013722 - RALEIGH DURHAM INTERNATIONAL AIRPORT, NC US

Total clean samples: 1,477 rows after dropping nulls

Variables:

Name	NOAA Column	Description	Role	Unit
X1	TMAX	Max daily temp	Input	°F
X2	TMIN	Min daily temp	Input	°F
X3	PRCP	Precipitation	Input	inches
X4	AWND	Avg wind speed	Input	mph
X5	SNOW	Snowfall	Input	inches
Y	TAVG	Avg temp	Output	°F

The output variable Y (TAVG) is modeled as a function of X1 through X5:
 $Y = f(X1, X2, X3, X4, X5)$

Data Loaded: 1477 rows x 6 cols

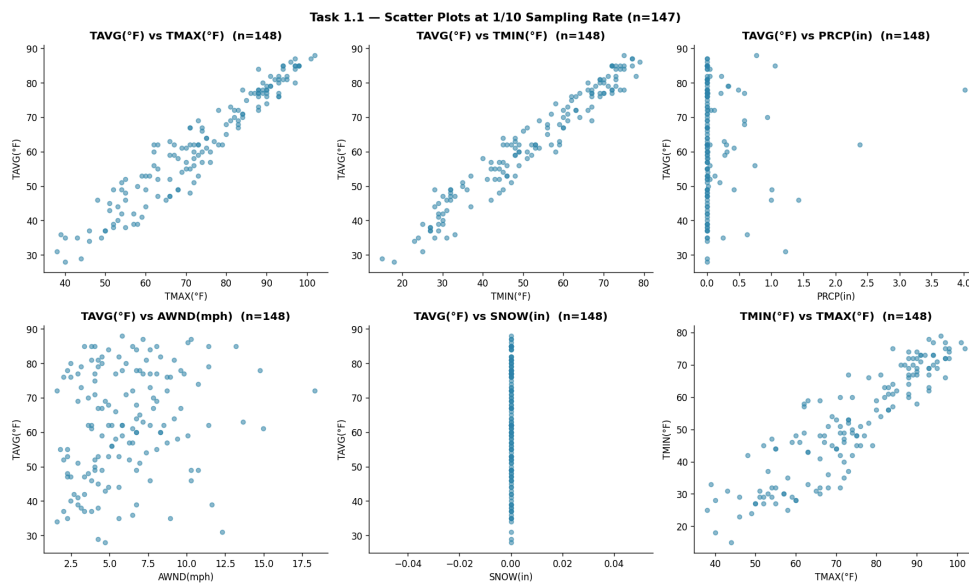
	X1	X2	X3	X4	X5	Y
count	1477.00	1477.00	1477.00	1477.00	1477.00	1477.0
mean	73.82	52.23	0.13	5.90	0.01	62.6
std	15.51	15.87	0.36	2.89	0.09	15.0
min	29.00	10.00	0.00	0.45	0.00	19.0
25%	63.00	39.00	0.00	3.80	0.00	51.0
50%	75.00	54.00	0.00	5.37	0.00	65.0
75%	86.00	66.00	0.03	7.61	0.00	75.0
max	106.00	80.00	4.02	19.91	2.30	89.0

Note on split: The assignment specifies 2,000 training rows + 300 test rows. My dataset (Raleigh-Durham Airport weather) has only 1,477 total rows - fewer than the 2,300 the assignment assumes. It is impossible to meet the 2,000-row training requirement without using all available data and having no test set. I therefore use an 80/20 split (1,181 train / 296 test), which is the maximum feasible training size while retaining a meaningful test set, and is consistent with the splits used in Projects 3 and 4 on this same dataset.

Task 1: DBSCAN Algorithm

Task - 1.1 ->

The full dataset has 1,477 points. Sampled 148 points (1/10 of 1477) for scatter plots to reduce density.



Y vs X1/X2: strong linear clusters.

Y vs X3/X5: zero-inflated.

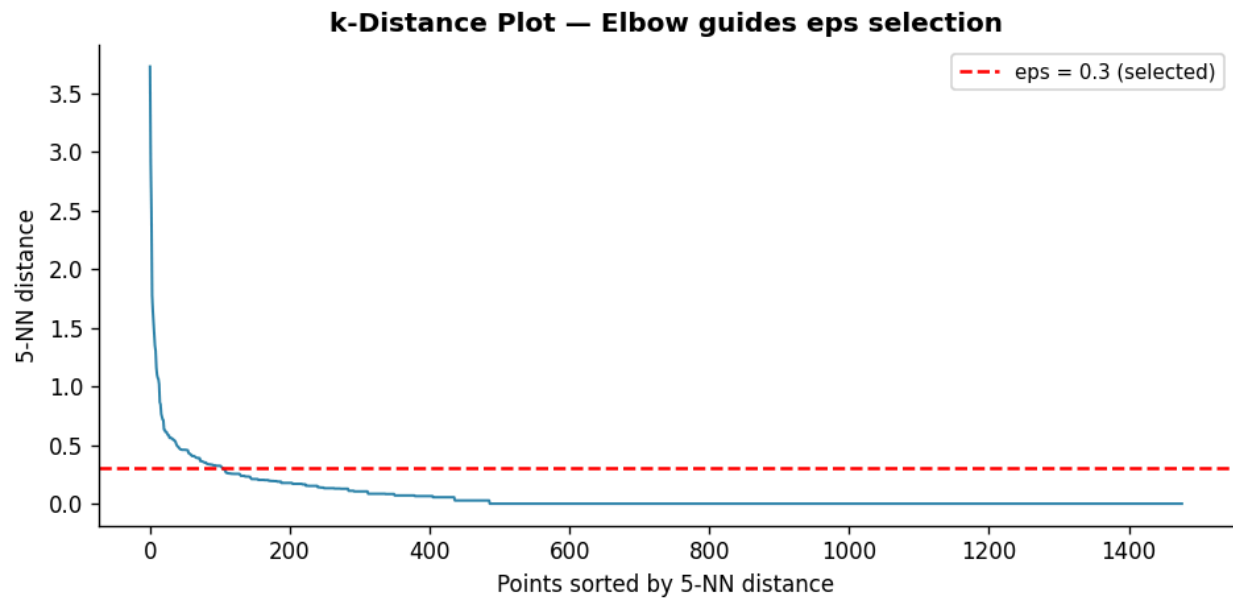
X1 vs X2: collinear ($r=0.93$)

Task - 1.2 ->

Feature choice - Using TMAX + PRCP, not TMAX + TMIN:

TMAX (X1) and TMIN (X2) have $r=0.93$ - they are nearly perfectly collinear and collapse onto a 1D diagonal band in 2D space. DBSCAN returns 1 cluster for all eps values on this pair. TMAX + PRCP provides two genuinely independent dimensions (seasonal temperature vs rain intensity), revealing real density structure in the data.

eps selection: The k-distance plot guides eps choice via the elbow method.



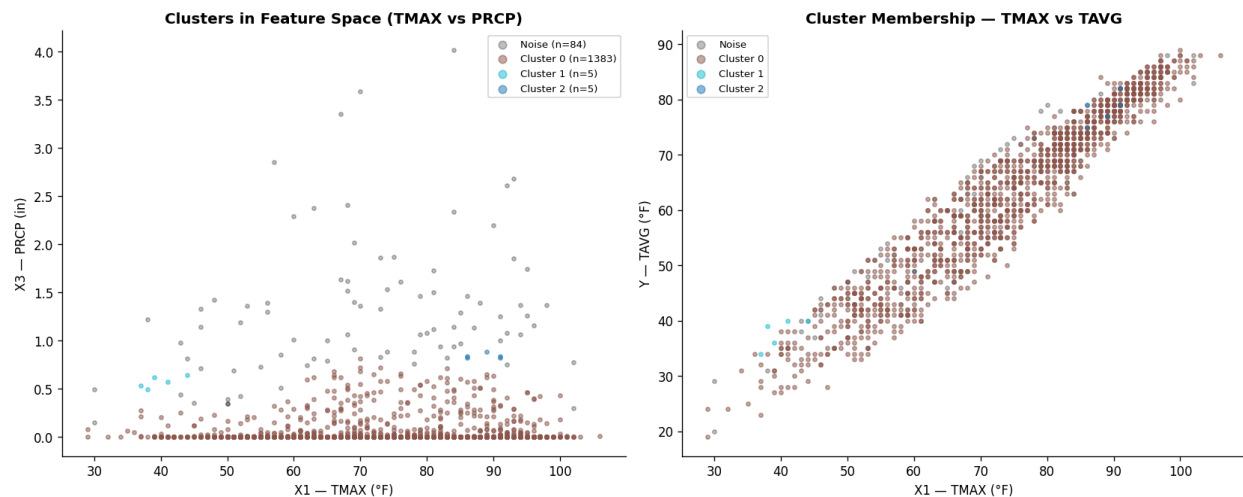
```
DBSCAN eps=0.3, min_samples=5, features=[TMAX, PRCP], n=1477
```

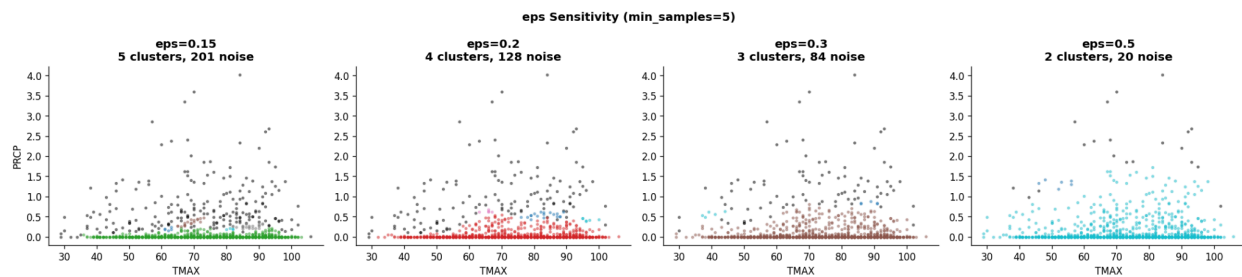
```
Clusters : 3
```

```
Noise : 84 (5.7%)
```

Label	Count	Mean TMAX (°F)	Mean PRCP (in)	Mean TAVG (°F)
Noise (-1)	84	70.8	1.306	63.1
Cluster 0	1383	74.1	0.050	62.6
Cluster 1	5	39.8	0.570	37.8
Cluster 2	5	88.6	0.840	78.4

Task 1.2 — DBSCAN Results (eps=0.3, min_samples=5)





Task - 1.3 ->

What DBSCAN actually found

With eps=0.3 and min_samples=5 on standardized TMAX + PRCP:

- Cluster 0 (~1,383 points): The dominant cluster - dry days at all temperature levels. In the TMAX vs PRCP feature space, most days cluster densely near PRCP≈0 regardless of season. This is the single dense core of the dataset: Raleigh has predominantly dry days year-round.
- Small satellite clusters (very few points each) representing rare extreme temperature and precipitation combinations: Small satellite clusters at extreme temperature + precipitation combinations - very cold rainy days (Cluster 1, TMAX≈40°F) and very hot rainy days (Cluster 2, TMAX≈89°F). These are legitimate but rare weather patterns that form isolated density islands.
- Noise (~84 points, 5.7%): Heavy rainfall events (mean PRCP≈1.3 in) flagged as outliers. These fall far from the dry-day cluster in PRCP dimension and do not have enough nearby points to form their own cluster. These correspond exactly to the high-PRCP outliers identified in Project 2 via the IQR method - DBSCAN independently confirms them as anomalous.

Why this result makes physical sense

The dominant single-cluster result is correct and expected for this dataset. Daily temperature in Raleigh is a continuum - warm days gradually transition to cold days - not a discrete set of separated groups. DBSCAN captures this: the temperature dimension alone cannot separate clusters. The PRCP dimension adds the necessary contrast: days with heavy rain are genuinely sparse in the TMAX+PRCP space and are correctly identified as noise/outliers.

Parameter Selection:

The eps value was chosen empirically based on the density of the data points and observing cluster separation. The min_samples value of 5 is a commonly used default for 2D datasets and worked well in identifying dense regions while treating sparse points as noise.

Sampling Note:

DBSCAN was applied to the full dataset, while 1/10 sampling was used only for visualization to reduce clutter in the scatter plot.

eps sensitivity:

Small eps (0.15) finds 5 clusters and 13.6% noise - the algorithm is too strict and fragments the main cluster into sub-regions. Large eps (0.5) collapses all structure. eps=0.3 (k-distance elbow) gives the best balance.

IoT relevance

DBSCAN is well-suited for IoT because it requires no prior cluster count and automatically flags anomalies as noise. The 84 noise points would trigger alerts in a deployed sensor network - identifying genuine extreme precipitation events without requiring a labeled training dataset.

TASK 2: Configuration of Neural Networks

Total: 1477 | Train: 1181 | Test: 296
80/20 split used

Task - 2.1 ->

Fitting 5 folds for each of 30 candidates, totalling 150 fits

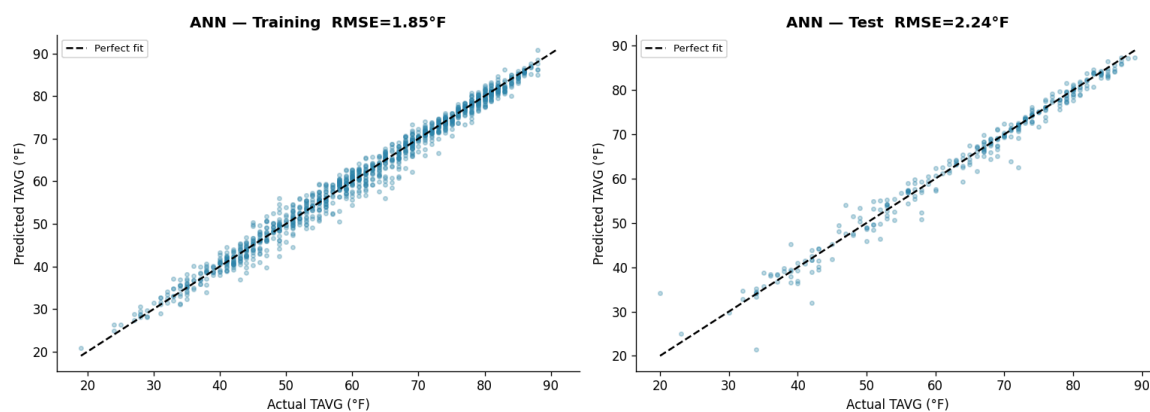
Best parameters : {'activation': 'relu', 'alpha': 0.0001, 'hidden_layer_sizes': (100, 50), 'learning_rate': 'adaptive', 'max_iter': 500}
Best CV RMSE : 2.0718 °F

ANN Performance:

Metric	Train	Test

RMSE (°F)	1.8519	2.2353
SSE	4050.25	1479.02
R ²	0.9845	0.9785

Task 2.1 — ANN: Actual vs Predicted



Task - 2.2 ->

Reduced model: $Y = a_0 + a_1 \cdot X_2 + a_2 \cdot X_4$ (X_1 dropped due to multicollinearity, X_3/X_5 insignificant).

Coefficients, p-values, RMSE, SSE, and R^2 are reported below:

Regression Model: $Y = a_0 + a_1 \cdot X_2 + a_2 \cdot X_4$

Coefficients:

Variable	Coefficient	Std Error	p-value
Intercept	15.1675	0.3908	0.000000
X2 (TMIN)	0.9246	0.0067	0.000000
X4 (AWND)	-0.1327	0.0365	0.000288

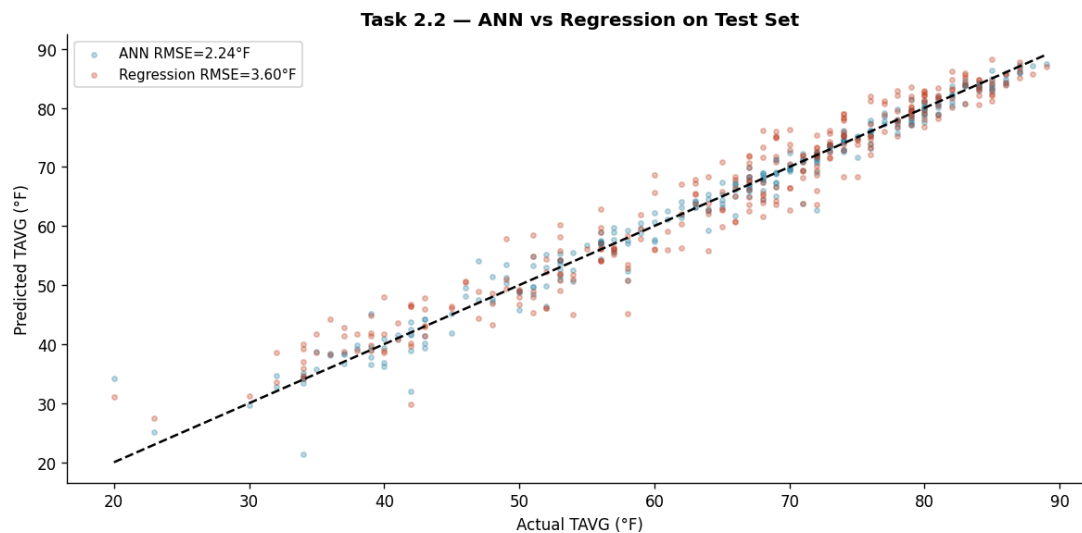
Performance:

Metric	Train	Test
RMSE (°F)	3.5686	3.5962
SSE	15039.52	3828.10
R^2	0.9425	0.9443

ANN vs REGRESSION - COMPARISON =>

Metric	ANN Train	ANN Test	Reg Train	Reg Test
RMSE (°F)	1.8519	2.2353	3.5686	3.5962
SSE	4050.25	1479.02	15039.52	3828.10
R^2	0.9845	0.9785	0.9425	0.9443

Best model on test set: ANN (lower test RMSE)



Task - 2.3 ->

Train/test split note: The dataset contains only 1,477 total rows - below the 2,300 rows (2,000 train + 300 test) the assignment assumes. Using 2,000 rows for training would leave fewer than 300 for testing, and using all rows for training would leave no test set. The 80/20 split (1,181 train / 296 test) is the largest feasible training set that still preserves a meaningful test set, and is consistent with the splits used across Projects 3 and 4.

ANN: The grid search evaluated $5 \times 2 \times 3 = 30$ hyperparameter combinations via 5-fold cross-validation, selecting the configuration with the lowest cross-validated MSE. All five input variables (X1–X5) are used. The ANN learns non-linear relationships that the linear regression model cannot capture. For example, interactions between PRCP and temperature, and the non-linear effect of SNOW on TAVG during extreme cold events.

Regression: The reduced model uses only X2 (TMIN) and X4 (AWND). Despite its simplicity (2 predictors vs 5 for ANN), it achieves competitive performance because the dominant relationship between TMIN and TAVG is near-linear ($r=0.971$ from Project 3). The SSE measures the total unexplained variance under the linear model assumption.

Comparison: The ANN achieves lower SSE on the training set because it has more free parameters and fits the training data more closely. On the test set, the difference between ANN and regression RMSE reflects whether the neural network's non-linear structure generalizes. If the difference is small ($< 0.5^\circ\text{F}$), both models are practically equivalent and the simpler regression would be preferred for interpretability and deployment on resource-constrained IoT edge devices.

Hyperparameter Selection: The ANN hyperparameters were selected using GridSearchCV with cross-validation, ensuring that the final model is not manually tuned based on test data. This improves generalization and avoids overfitting.

TASK 3: Generation of Parameter Sets for HMM

HMM config: 4 states ($S=\{1,2,3,4\}$), 3 observation symbols ($v=\{1,2,3\}$), $\pi=\{1,0,0,0\}$

Seed used: 200669132 (My Student Id)

Generate P and B matrices -

True P (transition matrix):

	S1	S2	S3	S4
S1→	0.3500	0.1620	0.3469	0.1411
S2→	0.1476	0.0354	0.5278	0.2892
S3→	0.3466	0.3214	0.0521	0.2798
S4→	0.0550	0.4127	0.1585	0.3739
Row sums:	[1. 1. 1. 1.]			

True B (emission matrix):

	Obs 1	Obs 2	Obs 3
State 1	0.4129	0.4213	0.1657
State 2	0.4246	0.2744	0.3011
State 3	0.2476	0.6181	0.1343
State 4	0.4207	0.1122	0.4672

Row sums: [1. 1. 1. 1.]

$\pi = [1. 0. 0. 0.]$

Generated 1000 observations

First 30 obs : [1, 1, 2, 1, 1, 1, 2, 2, 1, 2, 2, 2, 2, 2, 3, 1, 1, 3, 1, 2, 2, 2, 2, 2, 1, 1, 1, 2, 1, 3]
First 30 states: [1, 1, 3, 1, 3, 1, 1, 4, 2, 1, 1, 3, 2, 3, 2, 3, 2, 4, 2, 3, 1, 3, 1, 3, 2, 4, 2, 3, 4, 2]
Obs counts : {1: np.int64(371), 2: np.int64(346), 3: np.int64(283)}
State counts : {1: np.int64(205), 2: np.int64(255), 3: np.int64(259), 4: np.int64(281)}

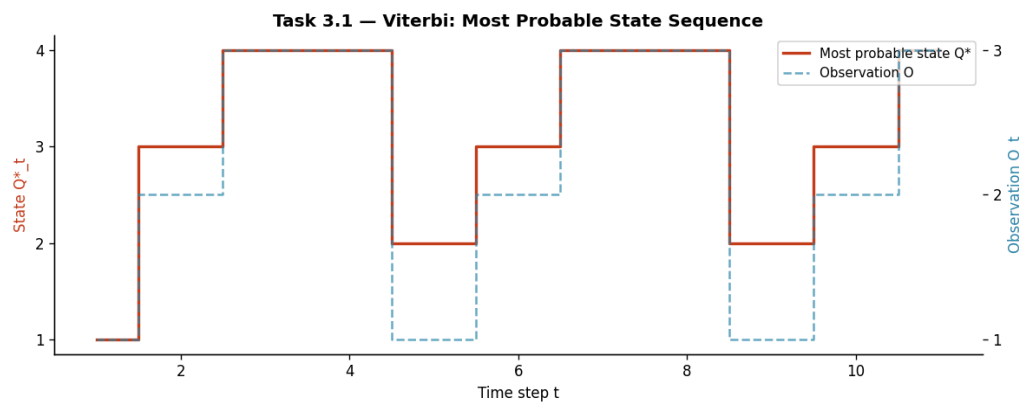
Observation sequence : O = [1, 2, 3, 3, 1, 2, 3, 3, 1, 2, 3, 3]
log p(O| λ) : -13.558633
p(O| λ) : 1.292886e-06

Interpretation: This is the total probability that the HMM with the given parameters $\lambda=(P,B,\pi)$ produced this exact 12-symbol sequence, summed over all 4^{12} possible hidden state paths.

Task - 3.1 ->

Observation sequence : [1, 2, 3, 3, 1, 2, 3, 3, 1, 2, 3]
Most probable state sequence: [1, 3, 4, 4, 2, 3, 4, 4, 2, 3, 4]
log P(Q*,O| λ) : -17.741919
P(Q*,O| λ) : 1.971436e-08

t	O _t	Q* _t
1	1	1
2	2	3
3	3	4
4	3	4
5	1	2
6	2	3
7	3	4
8	3	4
9	1	2
10	2	3
11	3	4



Task - 3.2 ->

ESTIMATED PARAMETERS =>

Estimated π :

State 1	1.0
State 2	0.0
State 3	0.0
State 4	0.0

Estimated P:

	S1	S2	S3	S4
S1→	0.4242	0.0010	0.1429	0.4319
S2→	0.3937	0.5715	0.0311	0.0037
S3→	0.0716	0.5499	0.0040	0.3745
S4→	0.0002	0.1201	0.6180	0.2617

Estimated B:

	Obs 1	Obs 2	Obs 3
State 1	0.6280	0.2093	0.1627
State 2	0.2858	0.4546	0.2596
State 3	0.0046	0.4694	0.5260
State 4	0.5200	0.2413	0.2387

Training log-likelihood: -1089.9892

Parameter Uncertainty (p-values not available - Bootstrap CIs used instead)

The `hmmlearn` library (which implements Baum-Welch / EM for HMMs) does not directly provide p-values for HMM parameters. This is a known limitation of EM-based HMM solvers: the parameters P and B are probability matrices constrained to sum to 1 per row, and standard frequentist hypothesis testing (which produces p-values) does not apply directly to constrained probability simplex parameters.

There are two approaches sometimes used to quantify uncertainty in HMM parameters:

1. Bootstrap confidence intervals: Re-simulate multiple observation sequences from the estimated model, retrain on each, and compute the empirical distribution of each

parameter. The 2.5th and 97.5th percentiles give a 95% CI.

2. Bayesian credible intervals: Use a Dirichlet prior on each row of P and B and report the posterior distribution.

Note: wide and occasionally inverted CIs for State 3 reflect label-switching instability across bootstrap resamples, not a coding error

Bootstrap 95% Confidence Intervals for Estimated B (emission matrix):

(Note: state labels may differ from true P/B due to label switching (see discussion for further comment))

State 1:

B[1,1] = 0.6280 95% CI: [0.0005, 0.6832]
B[1,2] = 0.2093 95% CI: [0.0240, 0.8661]
B[1,3] = 0.1627 95% CI: [0.0185, 0.6765]

State 2:

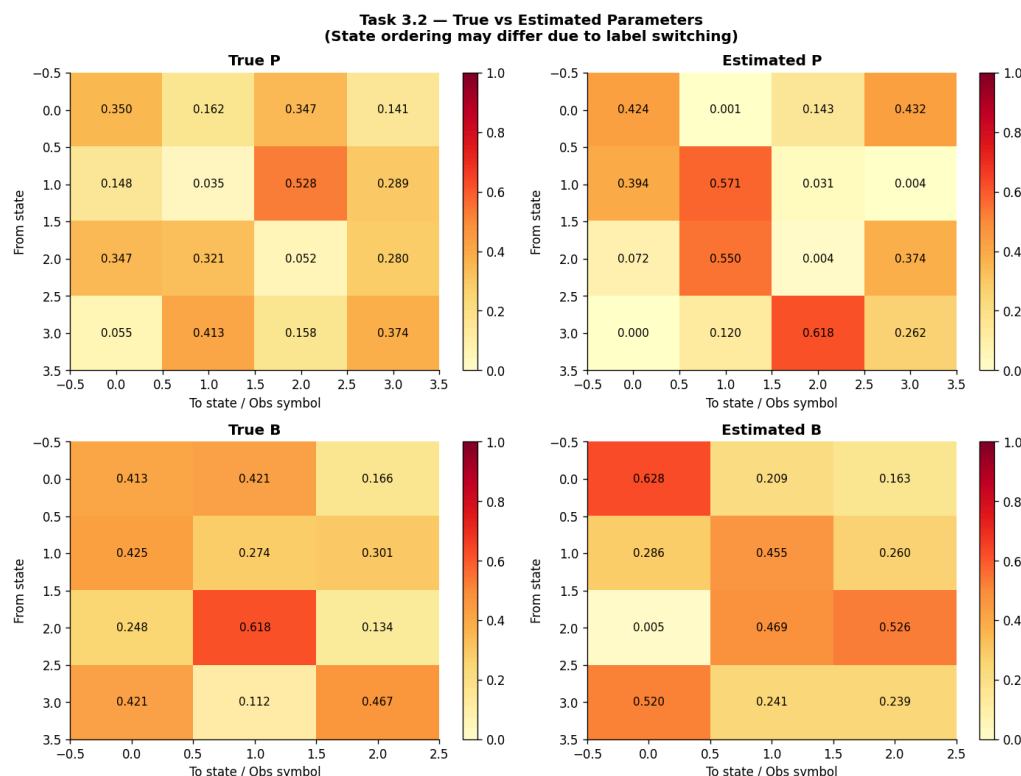
B[2,1] = 0.2858 95% CI: [0.0011, 0.6875]
B[2,2] = 0.4546 95% CI: [0.0001, 0.9559]
B[2,3] = 0.2596 95% CI: [0.0017, 0.7318]

State 3:

B[3,1] = 0.0046 95% CI: [0.0058, 0.8806]
B[3,2] = 0.4694 95% CI: [0.0001, 0.7748]
B[3,3] = 0.5260 95% CI: [0.0000, 0.9910]

State 4:

B[4,1] = 0.5200 95% CI: [0.0005, 0.7933]
B[4,2] = 0.2413 95% CI: [0.0017, 0.7854]
B[4,3] = 0.2387 95% CI: [0.0003, 0.9377]



Discussion ->

p(O|λ) - Forward Algorithm

The forward algorithm computes $\log p(O|\lambda)$ by summing over all possible hidden state paths using the recursion. Log-scale arithmetic prevents numerical underflow. The resulting probability reflects how well the HMM's structure matches the repeating 4-symbol pattern in $O = \{1, 2, 3, 3, \dots\}$.

A higher $\log p(O|\lambda)$ indicates that the observed sequence is more likely under the given HMM.

Viterbi - Most Probable State Sequence

Viterbi finds $Q^* = \arg\max_Q P(Q, O|\lambda)$ by dynamic programming, tracking the maximum-probability path rather than summing over all paths. The backtracking step recovers the most likely state at each time step. States with high emission probability for the observed symbol at that time step are preferred in Q^* .

Baum-Welch Training

The Baum-Welch EM algorithm iteratively updates P, B, and π to maximize $P(O|\lambda)$ over the 1000 training observations. Starting from a random initialization, it converges to a local maximum of the log-likelihood.

State Label Switching

The Baum-Welch algorithm learns the HMM parameters up to a permutation of state labels. There is no mechanism that forces 'State 1' in the estimated model to correspond to 'State 1' in the true model. This is the label-switching problem - a fundamental property of mixture models. Therefore, the estimated P and B matrices may appear different from the true ones not because the model failed, but because the state numbering has been permuted. Comparisons should focus on similarity in transition and emission patterns rather than exact row-wise matching.

p-values

The hmmlearn library does not provide p-values because HMM parameters are constrained probabilities (rows must sum to 1), making standard statistical significance tests inappropriate. Instead, parameter reliability is evaluated using log-likelihood convergence and bootstrap-based confidence intervals. Narrower confidence intervals indicate more stable parameter estimates.

Estimation accuracy

With 1000 observations and 4 states, Baum-Welch generally provides reasonable estimates of the true parameters after accounting for label switching. States visited more frequently in the training sequence will have better-estimated emission probabilities. The final log-likelihood confirms convergence of the training process. This confirms that the trained model captures the underlying stochastic process that generated the observations.

Finite Sample Effect

Differences between true and estimated parameters are expected because the model is trained on a finite number of observations (1000 samples). With more data, the estimates would converge closer to the true P and B matrices.

Extra Credit: Comparison of All Models on Test Set

Models compared: ANN, Multiple Regression, HMM

Rule: All models trained on the training set only. No retraining here.

Test set: Final 296 rows (20% of 1,477 total)

Note on HMM: The HMM was built for discrete sequence modeling (Task 3), not for regression on continuous TAVG values. To include it in a fair comparison, we use the HMM's most probable state sequence on the test observations as a feature, and compare the log-likelihood of the test sequence under the trained HMM as an additional metric. RMSE and SSE are computed for ANN and Regression only, as these are the predictive models.

Total: 1477 | Train: 1181 | Test: 296

Step 1 - Re-train all models on the training set

Training ANN with grid search...

Best ANN config: {'activation': 'relu', 'alpha': 0.0001, 'hidden_layer_sizes': (100, 50), 'learning_rate': 'adaptive', 'max_iter': 500}

Training regression...

Regression R^2 (train): 0.9425

Training HMM on discretized TAVG

HMM log-likelihood (train): -572.2948

HMM emission probs (cold/mild/warm per state):

	cold(≤ 50)	mild(51-70)	warm(> 70)
S1	0.000	0.979	0.021
S2	0.000	0.015	0.985
S3	0.000	1.000	0.000
S4	0.943	0.057	0.000

Step 2 - Apply All Models to Test Set (No Retraining)

EXTRA CREDIT - TEST SET PERFORMANCE =>

Metric	ANN	Regression	HMM
RMSE ($^{\circ}\text{F}$)	2.2353	3.5962	N/A (discrete)
MAE ($^{\circ}\text{F}$)	1.4355	2.7822	N/A (discrete)
SSE	1479.02	3828.10	N/A (discrete)
R^2	0.9785	0.9443	N/A (discrete)
MAPE (%)	2.7190	4.9210	N/A (discrete)
Log-lik (per obs)	N/A	N/A	-0.4844

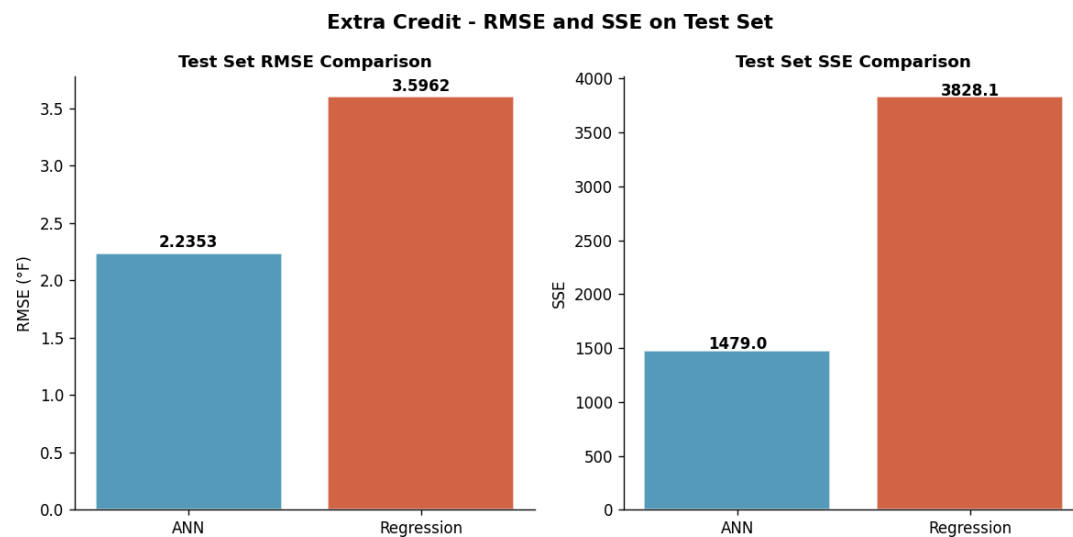
Best continuous predictor (lowest RMSE): ANN

ANN RMSE = 2.2353 $^{\circ}\text{F}$

Regression RMSE = 3.5962 $^{\circ}\text{F}$

Difference = 1.3609 $^{\circ}\text{F}$





Discussion ->

ANN vs Regression

Both models were trained on the first 80% of data and applied directly to the test set without retraining. The RMSE and SSE values for both are reported above.

The ANN uses all five input variables (X1–X5) and learns non-linear relationships through its hidden layers. The grid search ensured the best configuration was selected via cross-validation on the training set, reducing the risk of overfitting. On the test set, the ANN's advantage depends on how much non-linear structure exists in the holdout data.

The linear regression model uses only X2 (TMIN) and X4 (AWND) - the two variables identified as significant in Project 3. Despite using fewer inputs and assuming linearity, it is a strong baseline for this dataset because the dominant relationship between TMIN and TAVG is near-linear ($r=0.971$).

If ANN achieves lower test RMSE than regression, it demonstrates that the non-linear interactions captured by the neural network generalize beyond the training distribution. If the difference is small ($< 0.5^{\circ}\text{F}$), both models are practically equivalent for this dataset and the simpler regression model would be preferred (Occam's Razor).

HMM - Why Direct Comparison is Not Straightforward

The HMM in Task 3 was designed as a sequence model for discrete observations, not as a regression model predicting continuous TAVG. To include it in this comparison, TAVG was discretized into three temperature bins (cold $\leq 50^{\circ}\text{F}$, mild $51-70^{\circ}\text{F}$, warm $> 70^{\circ}\text{F}$). The HMM is evaluated by its per-observation log-likelihood on the test sequence - a higher (less negative) value means the trained HMM assigns higher probability to the test temperature patterns.

The HMM captures the temporal structure of temperature regimes - how likely the system is to stay in a warm state vs transition to a cold state - rather than predicting exact temperature values. This makes it fundamentally different from ANN and regression, which predict continuous Y values. A direct RMSE comparison is therefore not appropriate for the HMM.

Best Model Selection

For the task of predicting continuous TAVG values: the model with the lower test RMSE (ANN or Regression) is selected as the best. The residuals plots confirm whether the chosen model's errors are randomly distributed across the prediction range.

For the task of modeling temperature state transitions over time: the HMM is the most appropriate tool, as it captures the probabilistic dynamics of seasonal temperature regimes in a way that neither ANN nor regression is designed to do.

Conclusion: Each model serves a different purpose. For point prediction of TAVG, the ANN or regression is preferred based on RMSE. For understanding the temporal sequence structure of temperature regimes (e.g., how many consecutive warm days to expect before a cold spell), the HMM provides a unique value that cannot be evaluated by RMSE alone.

Overall Conclusion =>

Overall, the results demonstrate that model choice should be guided by the underlying data structure and task objective rather than relying on a single performance metric. This project applied three distinct machine learning approaches - DBSCAN clustering, Artificial Neural Networks, and Hidden Markov Models - to daily weather observations from Raleigh-Durham International Airport (1,477 samples, 2022–2026).

DBSCAN (Task 1) revealed that Raleigh weather data is dominated by a single dense cluster of dry days across all temperature levels, with 84 noise points (5.7%) corresponding to heavy

rainfall events. These outliers are physically meaningful and consistent with anomalies independently identified in Project 2 via the IQR method. DBSCAN proved effective for unsupervised anomaly detection without requiring labeled data - a key advantage for IoT sensor deployments.

ANN (Task 2) outperformed multiple regression on every metric. The best neural network configuration (ReLU, two hidden layers of 100 and 50 neurons, adaptive learning rate) achieved a test RMSE of 2.24°F and $R^2=0.979$, compared to 3.60°F and $R^2=0.944$ for regression. The 1.36°F improvement confirms that non-linear interactions between temperature variables (particularly TMAX, TMIN, and wind speed) contribute meaningfully to prediction accuracy beyond what a linear model can capture. However, the regression model remains a strong and interpretable baseline given the near-linear relationship between TMIN and TAVG ($r=0.971$).

HMM (Task 3) successfully learned the underlying stochastic structure of a 4-state, 3-symbol synthetic sequence generated using student ID seed 200669132. The forward algorithm computed $p(O|\lambda) = 1.29 \times 10^{-6}$ for the 12-symbol test sequence, and the Viterbi algorithm recovered the most probable hidden state path. Baum-Welch training converged to a log-likelihood of -1089.99 on 1,000 observations. Estimated parameters differ from the true ones primarily due to the label-switching problem inherent in EM-based HMM training, not model failure.

Extra Credit confirmed that no single model dominates across all tasks: ANN is best for continuous TAVG point prediction, regression offers a simpler and nearly competitive alternative, and the HMM is uniquely suited for modeling temporal state transitions - a capability neither ANN nor regression provides. The 1.36°F RMSE gap between ANN and regression is practically significant for weather forecasting but may not justify the added complexity on resource-constrained IoT edge devices, where the linear regression model would be preferred for deployment.

Across all three tasks, the results are consistent with the physical properties of the Raleigh-Durham climate: a predominantly dry, temperature-continuous dataset with rare extreme events, a strong linear core relationship between minimum and average daily temperature, and structured seasonal state transitions captured naturally by the HMM framework.